

Blockchains & Distributed Ledgers

Lecture 09

Damiano Abram



Slide credits: DA, Aggelos Kiayias, Michele Ciampi, Dimitris Karakostas

Security critical computations

- How to obtain the output of a security critical computation
- Deterministic with public inputs?
 - Repeat multiple times and consensus can be reached about its output
 - Example: blockchain systems with smart contracts
- What if it is probabilistic with public inputs?
 - Coin flipping protocol
- What if it uses private data?
 - Secure Multiparty Computation (MPC)

Secure Multiparty Computation and Applications

- Sharing responsibility for signatures and cryptographic keys
 - Secret sharing
- Security critical computations
 - Coin flipping and verifiable secret-sharing
 - Secure multiparty computation (MPC)
- Fair swaps and fair MPC

Secret sharing

Overarching question

- How to protect security critical operations?
- Key idea: share responsibility and somehow tolerate faulty participants
 - Cryptographic keys?
 - Cryptocurrency addresses?
 - Computations?
 - What about computations on private data?

Multi-sig transactions

- Multi-sig: a tx that can be redeemed if n parties sign it
- A payment to a script (P2SH) can facilitate a multi-signature transaction

```
scriptPubKey: OP_HASH160 <redeemscriptHash> OP_EQUAL
```

```
scriptSig: OP_0 <sig_Ai> ... <sig_An> <redeemscript>
```

```
redeemscript = OP_m <A1 pubkey> <A2 pubkey>... <An pubkey>  
<OP_n> <OP_CHECKMULTISIG>
```

Secret-Sharing

Main question:

- How to share a secret s to n shareholders so that:
 - Any subset including t of them can recover the secret
 - Any subset including *less than* t of them knows nothing about the secret
- Relative questions:
 - Can we solve this for any n and $t \leq n$?
 - What is the relation between the size of s and the size of each share?

Finite fields

- Finite sets equipped with *two operations*, behaving similarly to addition and multiplication over the real numbers (which is an infinite field)
- Finite fields exist with number of elements equal to p^k , for:
 - any prime number p
 - any positive integer k

Example. A binary finite field $\{0, 1\}$ with:

$$(a+b) \bmod 2$$

+	0	1
0	0	1
1	1	0

$$(a*b) \bmod 2$$

*	0	1
0	0	0
1	0	1

Secret-Sharing over a finite field

- Consider a secret x and N random values, subject to the constraint:

$$\sum_{i=1}^N x_i = x \quad (\text{over a finite field})$$

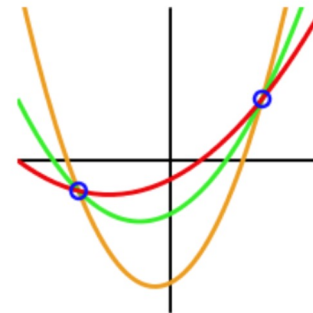
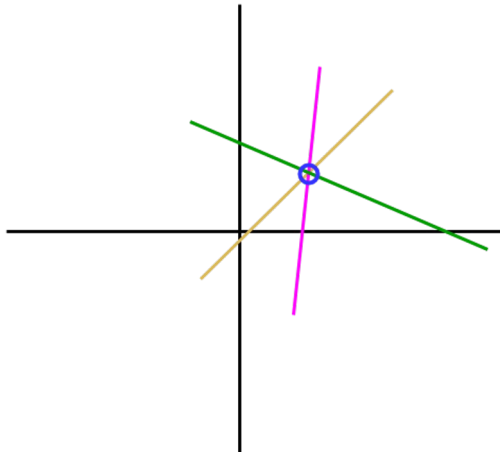
- This is called (additive) secret-sharing
- Knowledge of *any* $N-1$ values cannot be used to infer any information about x

Analysis

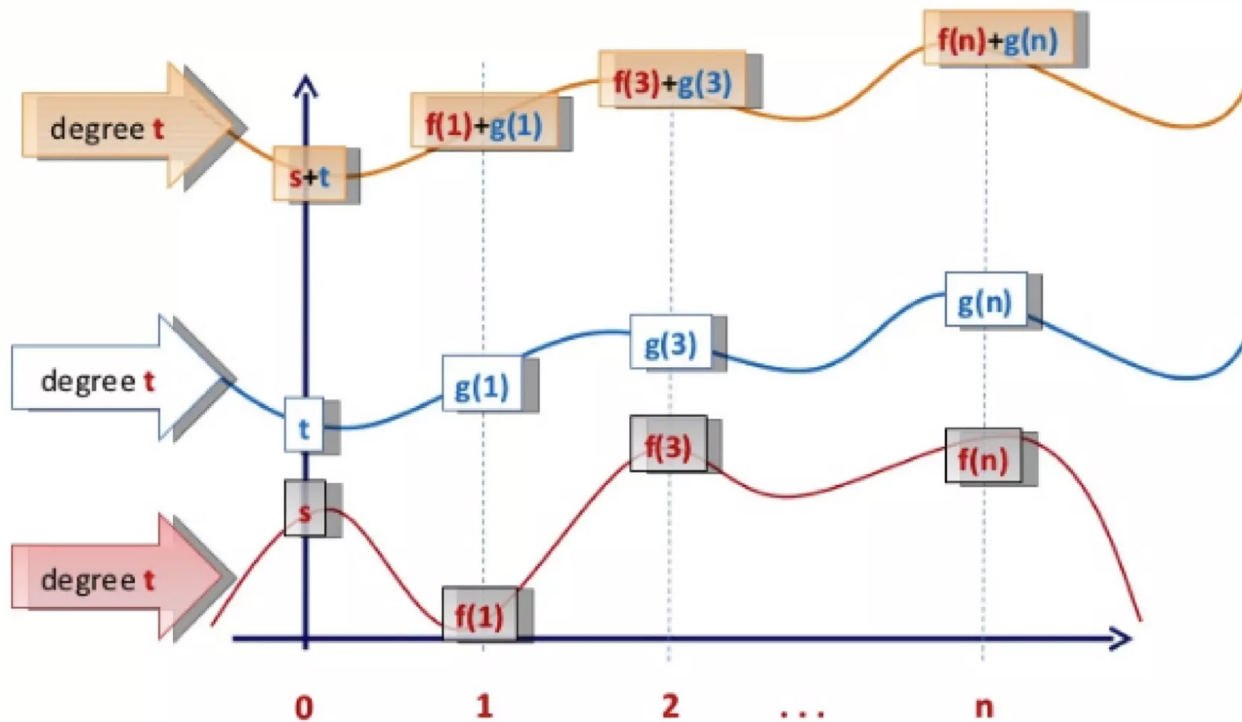
- Example: binary field
- If you hold only $N-1$ values $[x_2, \dots, x_N]$:
 - Two unknowns: x_1, s
 - One equation: $x_1 + x_2 + \dots + x_N = s$
- s cannot be undetermined
 - $s = 0 + x_2 + \dots + x_N$ (if $x_1 = 0$)
 - $s = 1 + x_2 + \dots + x_N$ (if $x_1 = 1$)

Generalisation t-out-of-n

- Consider a polynomial of degree d : $p(x) = a_0 + a_1x + \dots + a_dx^d$
- Any $d+1$ points of the polynomial completely determine it
- With d or less points, at least one degree of freedom remains
 - p cannot be fully determined
- We can use that idea to solve secret-sharing for any t, n



Generalisation t-out-of-n



Example

- 5 parties
- Polynomials of degree 2
- Any three parties (who hold 3 points) can interpolate such polynomials
- Any two parties have no information about the shared secret

Secret-sharing cryptographic keys

- Using polynomial secret-sharing, a cryptographic key can be split to multiple shareholders
 - Each shareholder gets a point on the plane
 - The secret/key is the solution to the polynomial problem
- Additional points to consider:
 - How should the value of t be determined:
 - in comparison to d ?
 - in comparison to n ?
 - To engage in the cryptographic operation, is it necessary to reconstruct the original key?
 - How to accomodate an evolving set of shareholders?

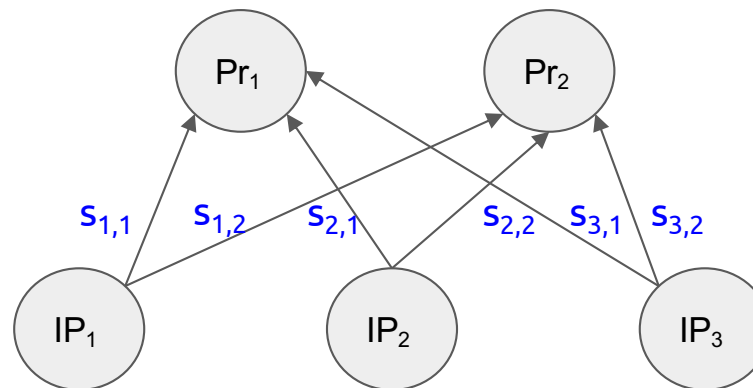
Secure MPC

Secure Multiparty Computation

- (Secure) Multiparty Computation (MPC)
- Parameterized by function f
- A set of n parties P_i contribute inputs $x_1, x_2, \dots, x_n$
- At the end of the protocol they compute $f(x_1, x_2, \dots, x_n)$
 - Everyone receives output $f(x_1, x_2, \dots, x_n)$
 - No party except P_i obtains information about x_i

MPC Construction Idea

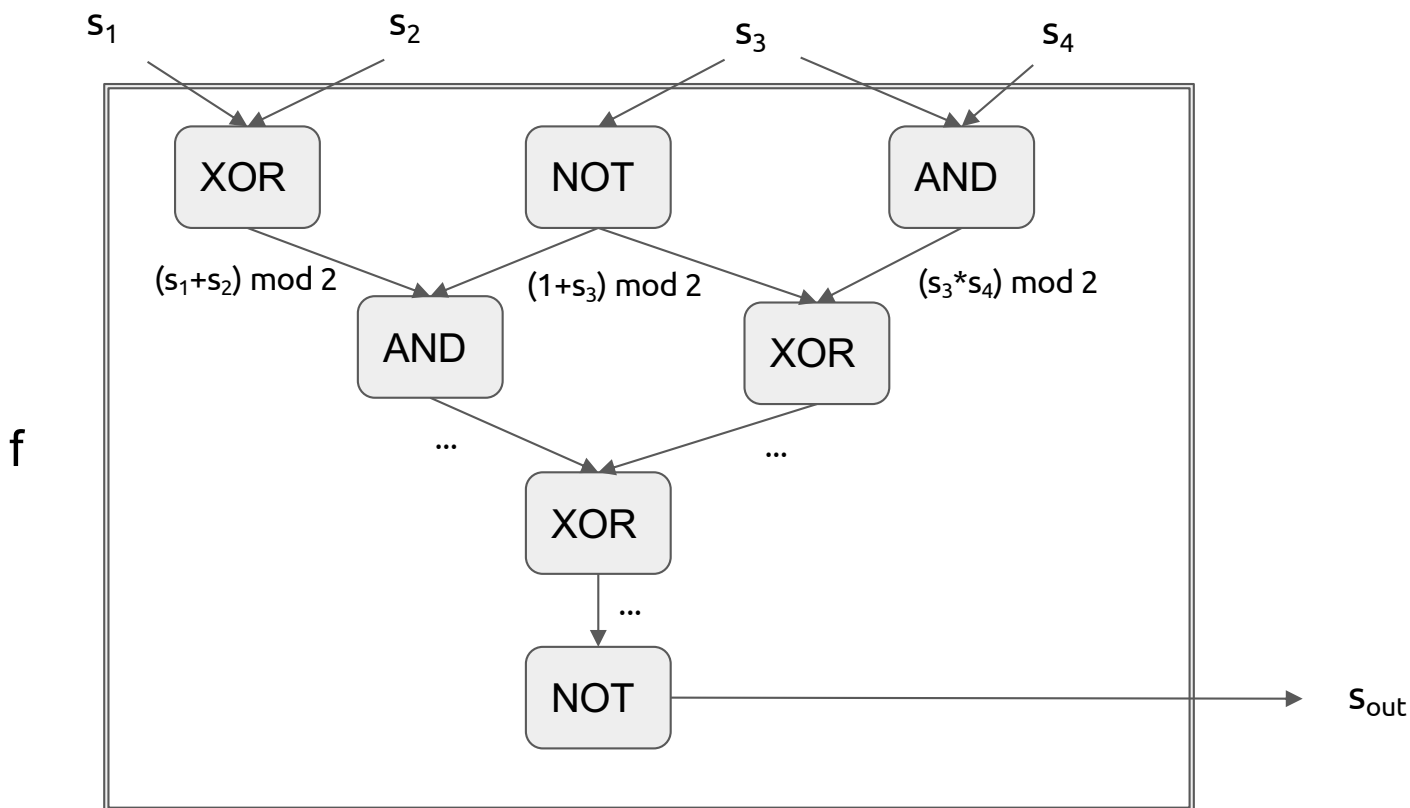
- Consider three roles
 - Input providers
 - Processors
 - Output-receivers
- Input providers secret-share their input to the processors
 - Additive secret-sharing



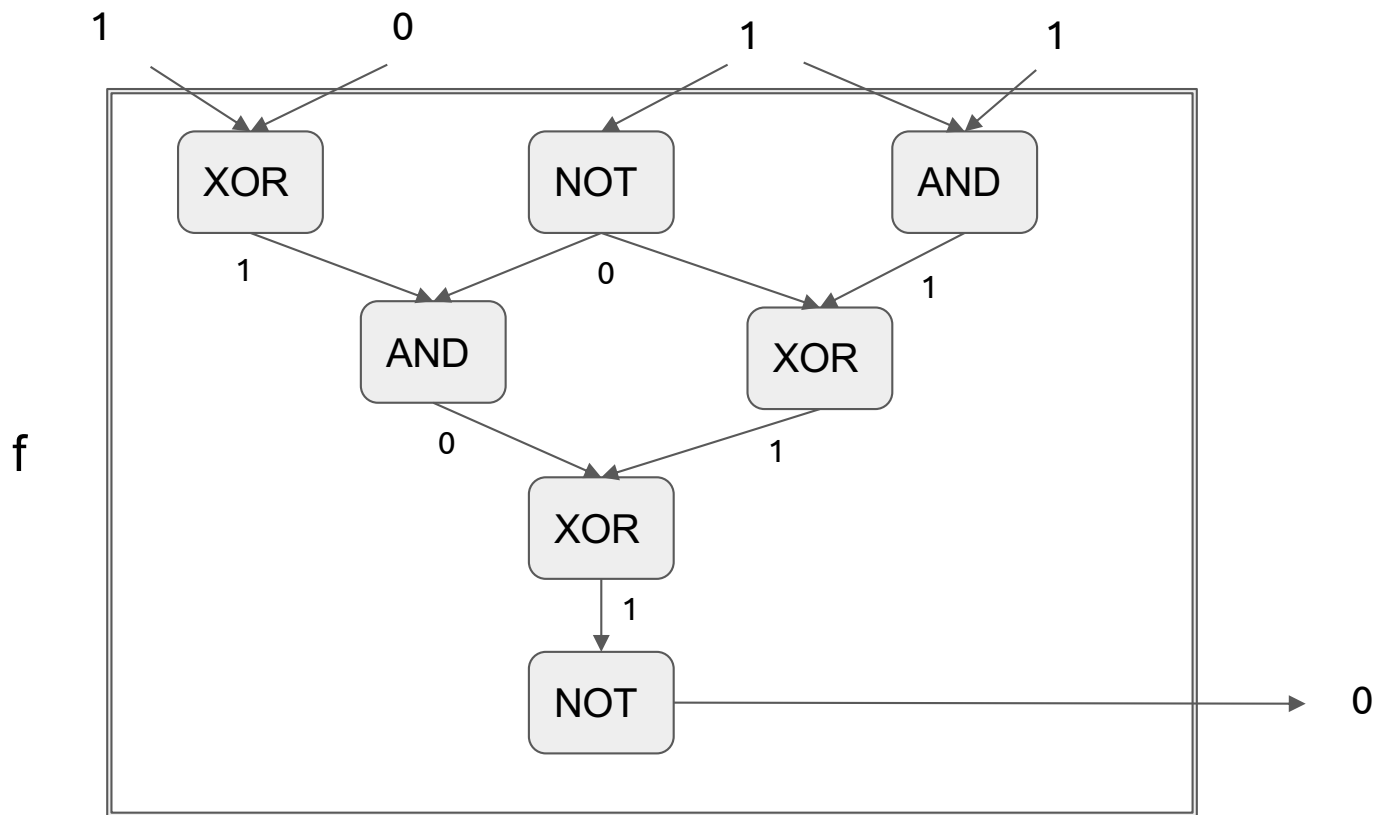
MPC Construction Idea

- Any function f can be expressed as a Boolean circuit
 - Fixed-size input
 - Upper-bound on number of steps (circuit depth)
 - Example: any boolean function can be implemented as a combination of NAND gates
- XOR, AND, NOT gates
- Arithmetic representation of gates
 - AND: Input: a, b ; Output: $(a*b) \bmod 2$
 - XOR: Input: a, b ; Output: $(a+b) \bmod 2$
 - NOT: Input: a ; Output: $(1+a) \bmod 2$
- Each processor executes the circuit with their shares as input
 - How to implement the gates s.t. operations on shares, when combined, produce the correct aggregate output?

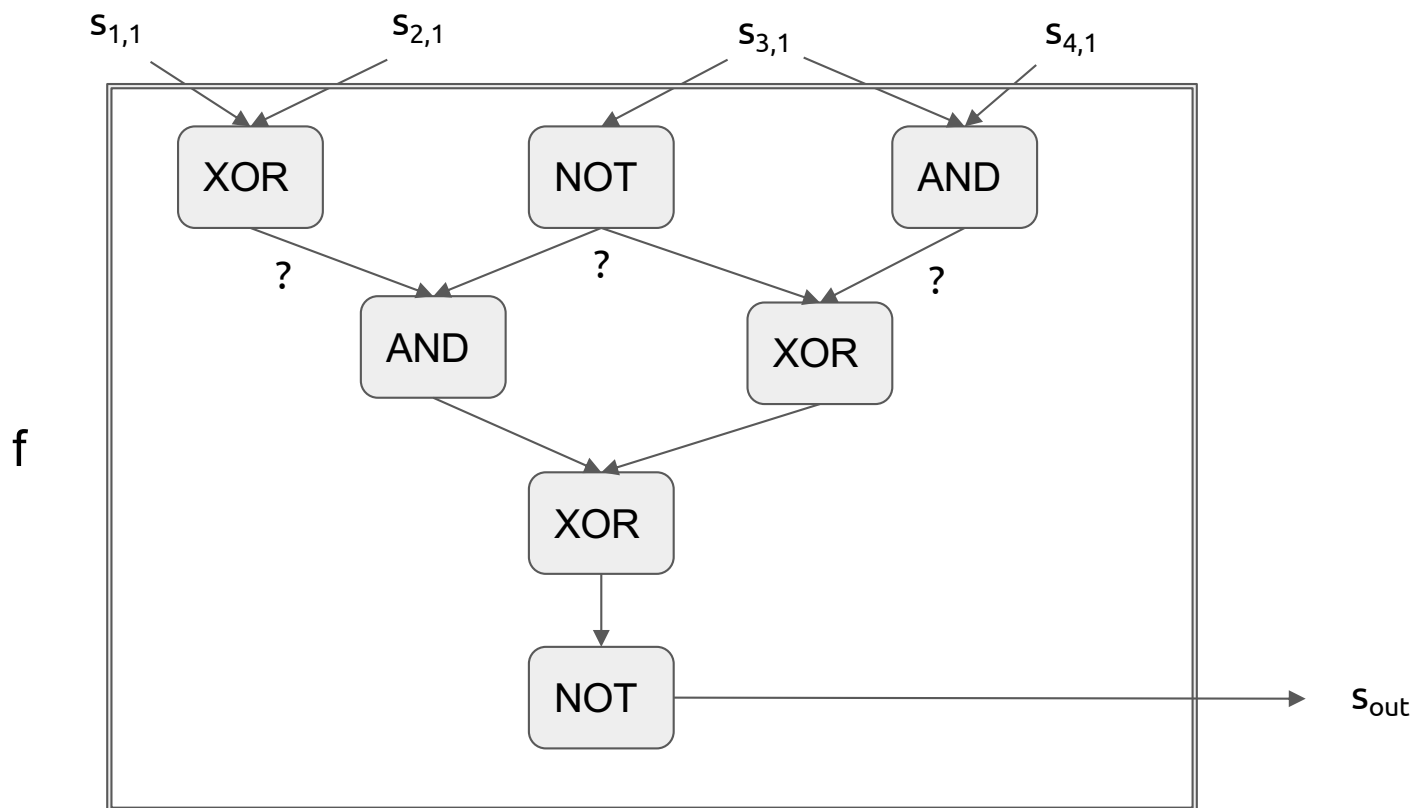
MPC Construction Idea, Example



MPC Construction Idea, Example



MPC Construction Idea, Example



MPC Construction Idea

NOT GATE

- Suppose m parties hold shares of two inputs to a NOT gate.

$$[a] = \langle a_1, \dots, a_m \rangle$$

- How do they calculate shares of the output of the NOT gate?

$$[\bar{a}] = \langle 1 + a_1 \bmod 2, a_2, \dots, a_m \rangle$$

MPC Construction Idea

XOR GATE

- Suppose m parties hold shares of two inputs to an XOR gate.

$$[a], [b] = \langle a_1, \dots, a_m \rangle, \langle b_1, \dots, b_m \rangle$$

- How do they calculate shares of the output of the XOR gate?

$$[a] + [b] \bmod 2$$

MPC Construction Idea

- Suppose m parties hold shares of two inputs to an AND gate.

AND GATE

$$[a], [b] = \langle a_1, \dots, a_m \rangle, \langle b_1, \dots, b_m \rangle$$

- How do they calculate shares of the output of the AND gate?

$$[a] \cdot [b] = \langle a_1 b_1 \bmod 2, \dots, a_m b_m \bmod 2 \rangle$$

but we want: $s_1 + \dots + s_m = \left(\sum_{i=1}^m a_i \right) \left(\sum_{i=1}^m b_i \right)$

MPC Construction Idea

- A Beaver triple is an initial secret-sharing of random values

$$x \cdot y = z$$

$$[x] = \langle x_1, \dots, x_m \rangle, [y] = \langle y_1, \dots, y_m \rangle, [z] = \langle z_1, \dots, z_m \rangle$$

AND GATE :

publish $d_i = a_i - x_i$ reconstruct d, e

$$e_i = b_i - y_i$$

define $s_i = de + dy_i + ex_i + z_i$ share calculation

$$\begin{aligned} \sum s_i &= de + d \sum y_i + e \sum x_i + xy \\ &= de + dy + ex + xy = (a - x)(b - y) + (a - x)y + (b - y)x + xy \\ &= ab \end{aligned}$$

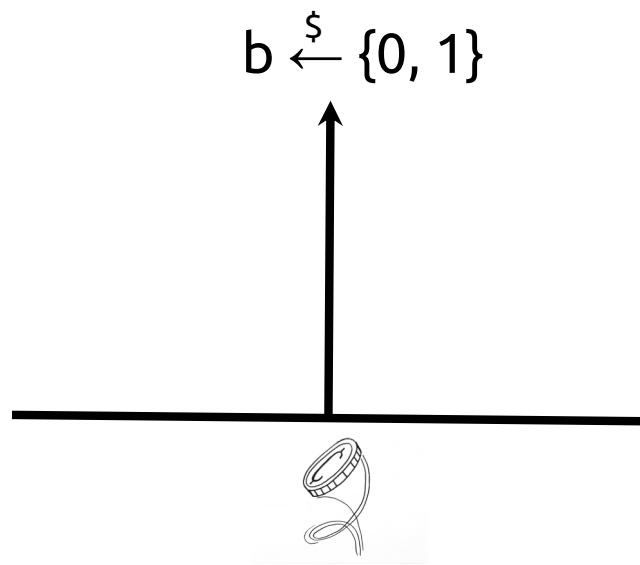
Constructing Beaver Triples

- The above construction idea requires the setup of all servers with a sufficient number of Beaver triples (how many?)
- Constructing Beaver triples can be done via special-purpose cryptographic protocols

MPC strengths and weaknesses

- Possible to compute any function f privately on parties' inputs
- Unless *honest majority* is present, there is no way to provide:
 - fairness: either all parties learn the output or none
 - guaranteed output delivery

Distributed Randomness Generation



Application: coin-flipping

- Alice and Bob want to flip a coin remotely
 - output a bit uniformly at random
- Alice doesn't trust Bob and vice versa
 - neither Alice nor Bob should be able to bias the bit choice

Application: coin-flipping

- Alice and Bob want to flip a coin remotely
 - output a bit uniformly at random
- Alice doesn't trust Bob and vice versa
 - neither Alice nor Bob should be able to bias the bit choice
- Solution:
 - Alice commits to a random coin
 - Bob commits to a random coin
 - Alice and Bob open the commitments
 - Output = XOR of (committed) values
- Consider:
 - Can the situation be improved in an N party coin flip?
 - What about when $>N/2$ parties are honest?
 - How do you deal with (selective) aborts?

A first step towards multi-party coin flipping

- Each player commits to their coin (publicly)
- Each player publishes a secret-sharing of the opening to their commitment
 - Any subset of at least $(N/2 + 1)$ players can reconstruct the opening
 - Shares should be encrypted with the respective public-keys of the parties
- If some parties abort the protocol: assuming that a subset of $>N/2$ parties continue, they can recover the share and terminate
- Any number of parties up to $N/2$ cannot gain any advantage over the honest parties

What if some parties announce incorrect shares?

- A secret cannot be retrieved from incorrect shares
- Selective aborts possible, as remaining parties cannot reconstruct the secret
- Possible solution: require that all commitments open at the end irrespectively of aborts
 - deviating players will be caught, but still have the option to selectively abort if they wish
 - other parties will only know of the abort when it is too late
- One possible approach: issue monetary penalties to those that abort

Publicly Verifiable Secret-sharing (PVSS)

- The dealer creates shares that are distributed in encrypted form
 - The shares can be **publicly verified** as correct
 - Verifiability should not leak information about the secret
-
- PVSS enables parties to detect improper share distribution at the onset
 - Protocol can still be aborted, but any abort would be independent of the (random) coin!

PVSS Design Challenges

- Assuming:

$$\sum_{i=1}^N x_i = x \quad \psi_i = \mathcal{E}_i(x_i)$$
$$\psi = \mathbf{Com}(x)$$

- Verify that the value encrypted in ψ_i satisfies the equation w.r.t. the values encrypted in ψ
- This problem can be solved using a **zero-knowledge proof**

Fairness

Workarounds for fairness

- Optimistic fairness (by involving a third party):
 - The protocol is basically not fair
 - A third party is guaranteed to be able to engage and amend the execution in case of deviation
- Gradual/timed release:
 - Protocols engage in many rounds
 - Parties gradually come closer to computing the output
 - “gradual closeness” can be measured in terms of:
 - probability of guessing the output
 - number of computational steps remaining to compute the output
 - Example:
 - At each round $l = 1, \dots, n$ the two parties can compute the output in 2^{n-l} steps
 - If a party aborts the interaction, the other party will be 2 times more steps “behind” in the calculation of the output

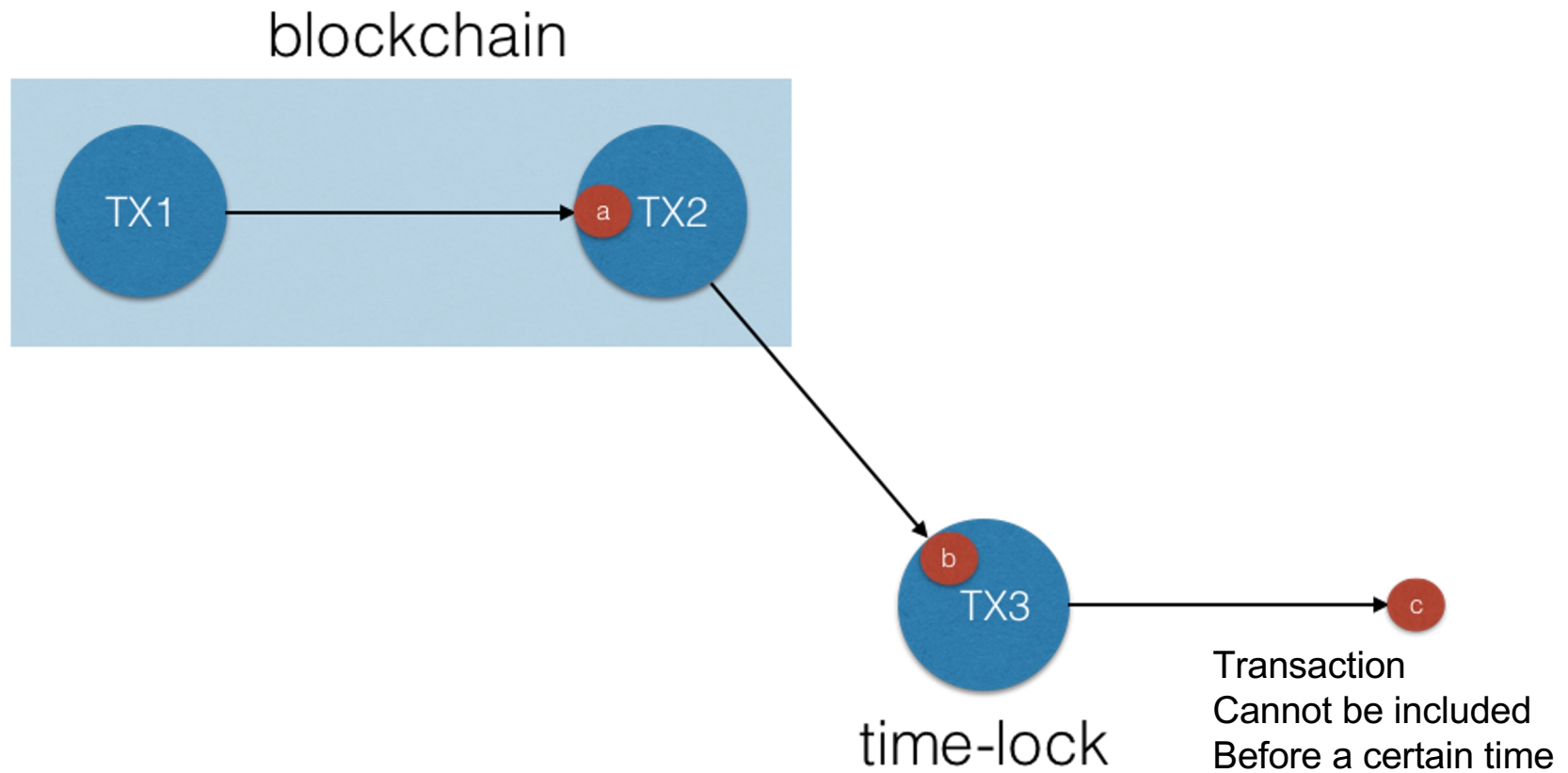
Using a blockchain

- Along the lines of optimistic fairness, but substituting the trusted third party with the blockchain
- How is that possible?
 - Blockchain cannot keep secrets
 - Rationale: penalize parties that deviate from the protocol

Basic tool: time-lock transactions

- Time-lock transactions
 - part of transaction data
 - specifies the earliest time that a transaction can be included in a block
- Key observation: if a conflicting transaction has already been included in the ledger, the time-lock transaction will be rejected

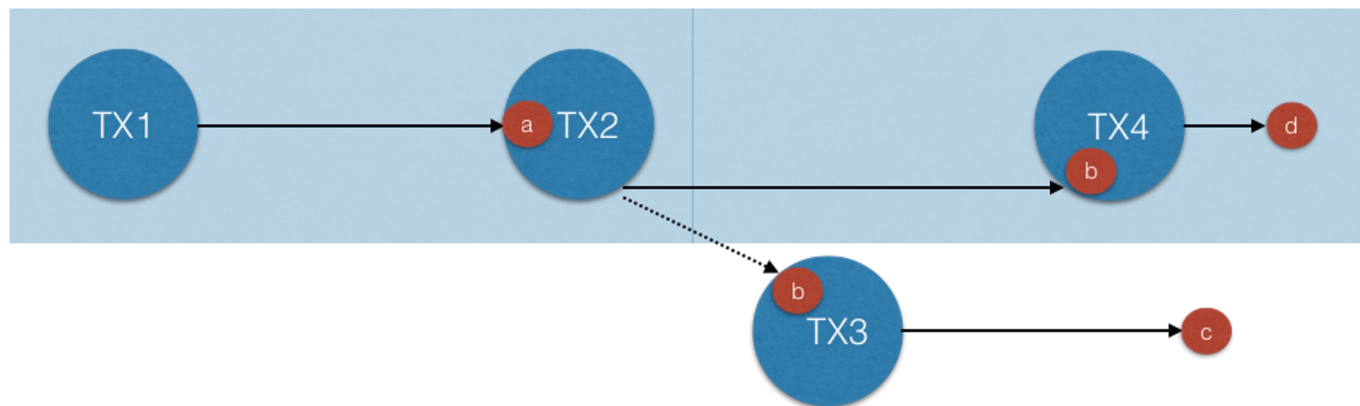
Time-lock example



Time-lock example



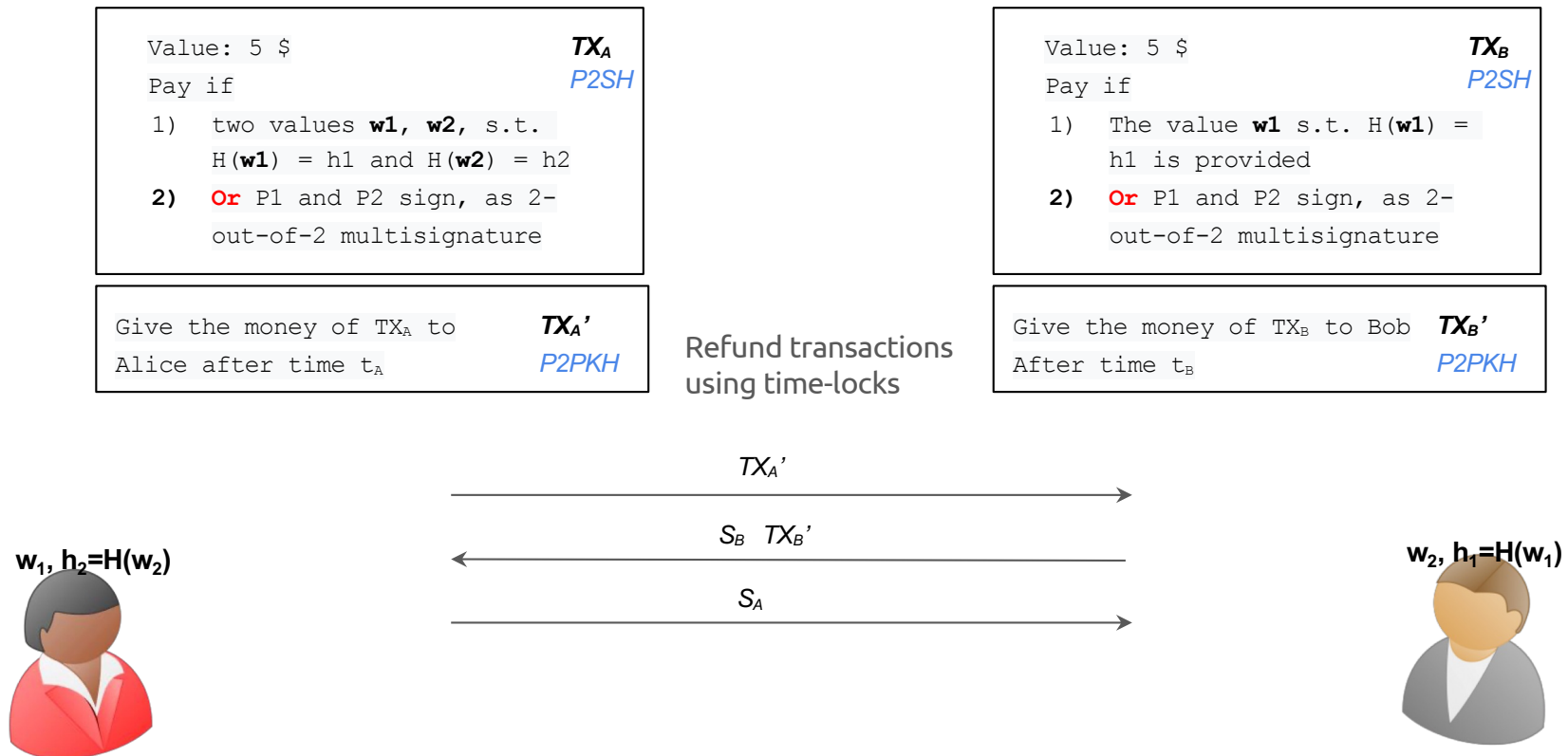
OR



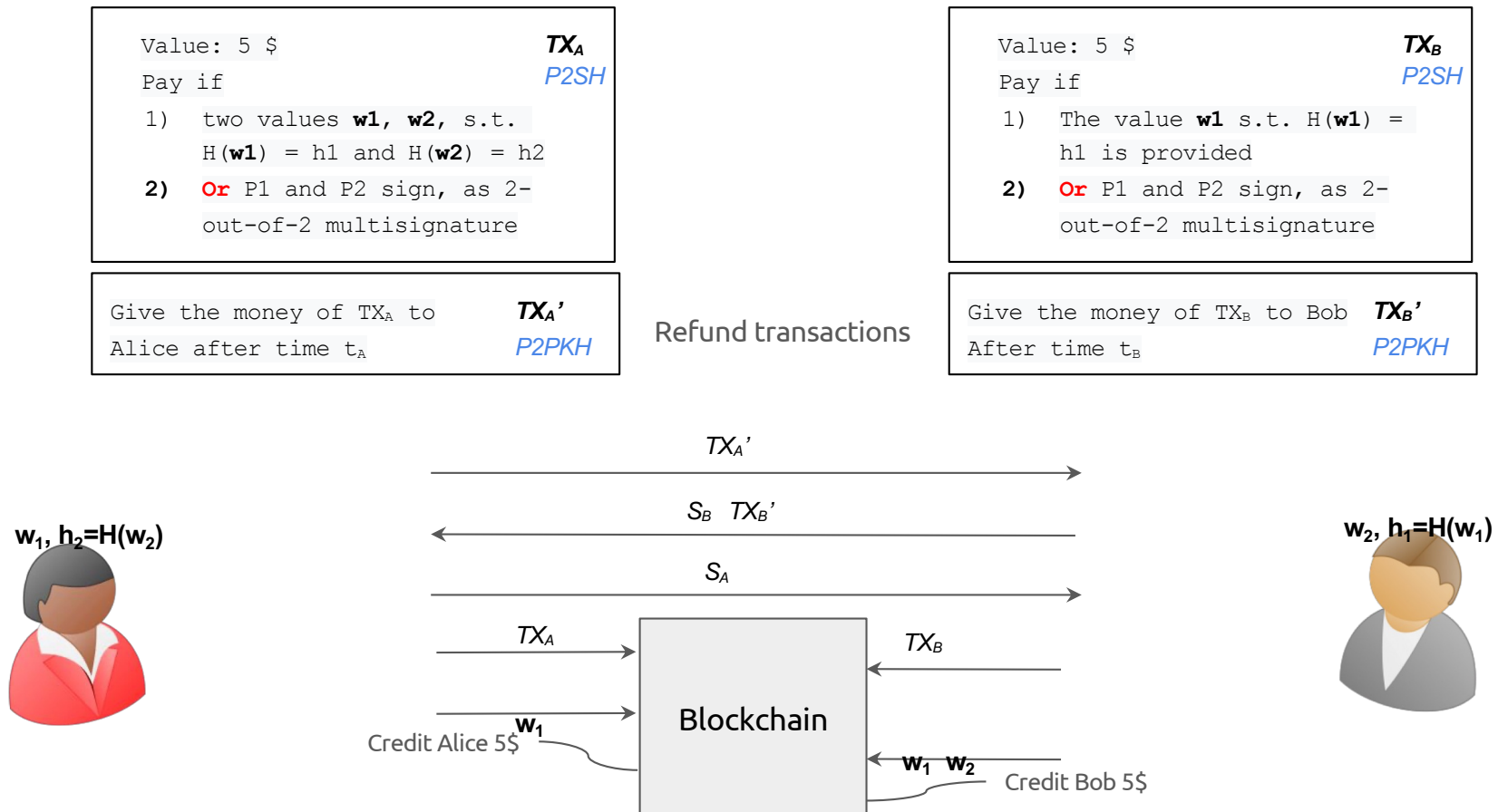
Fair swap of values using time-locks, Setup

- P_1 holds w_1 , $h_2 = H(w_2)$
- P_2 holds w_2 , $h_1 = H(w_1)$
- *They want to exchange w_1, w_2*

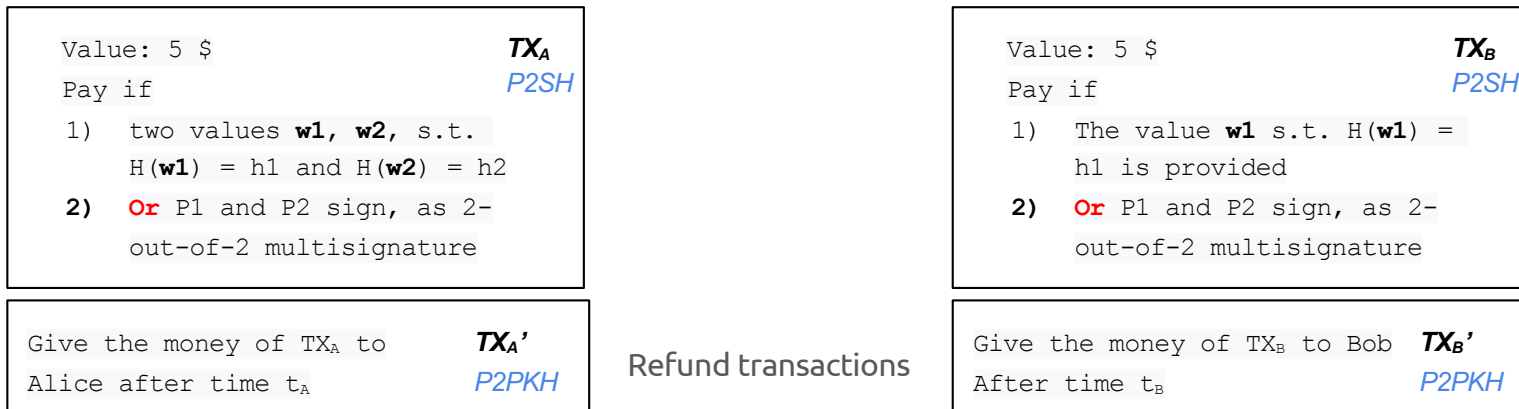
Fair swap of values using time-locks, Setup



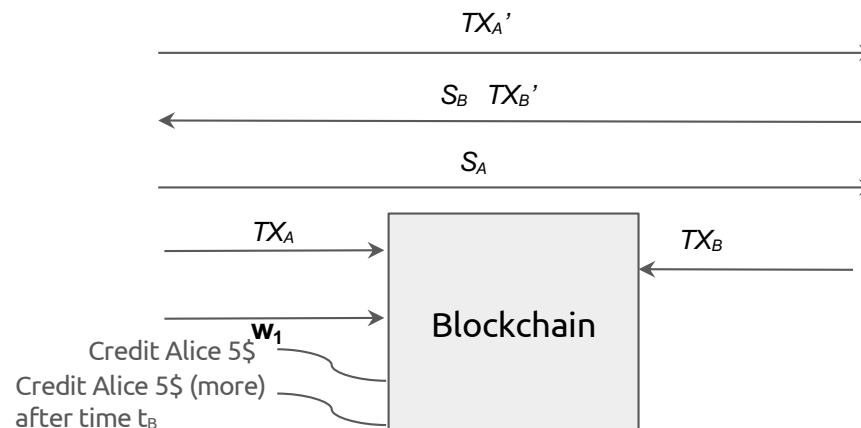
Fair swap of values using time-locks, Setup



Fair swap of values using time-locks, Setup



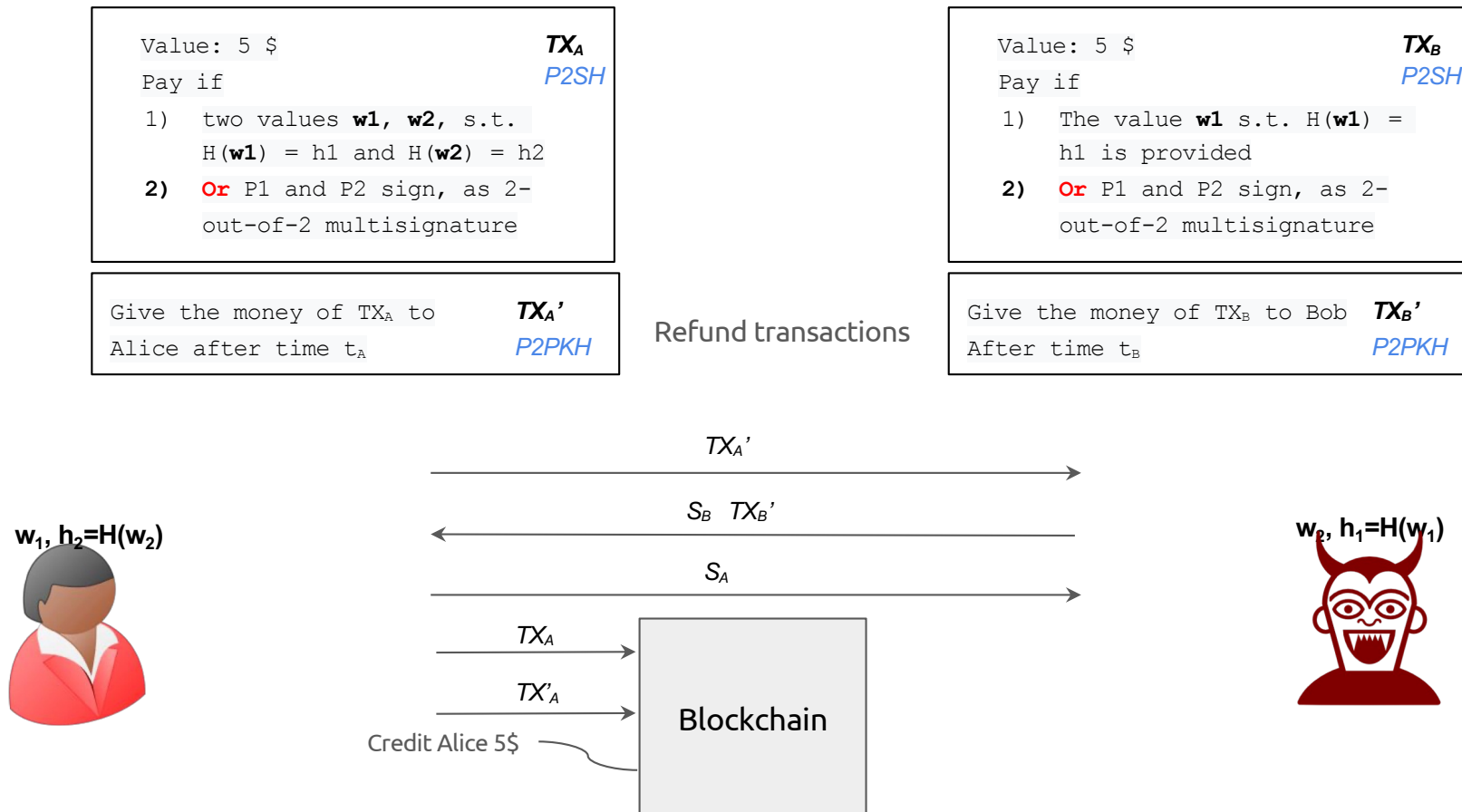
w₁, h₂=H(w₂)



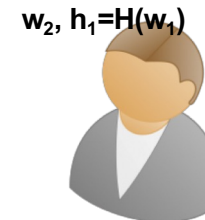
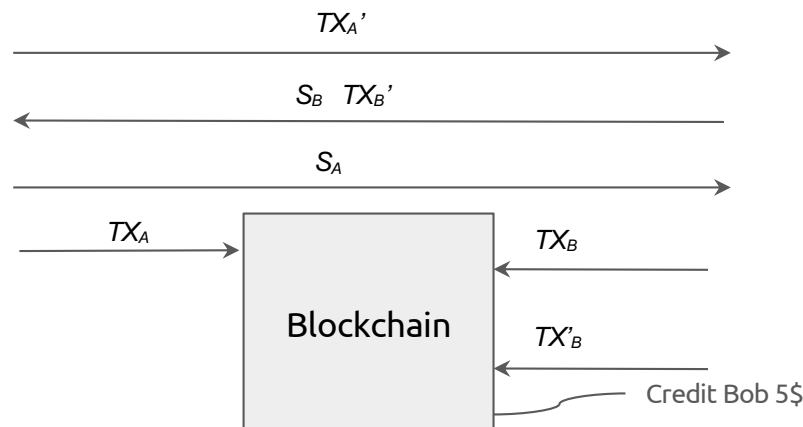
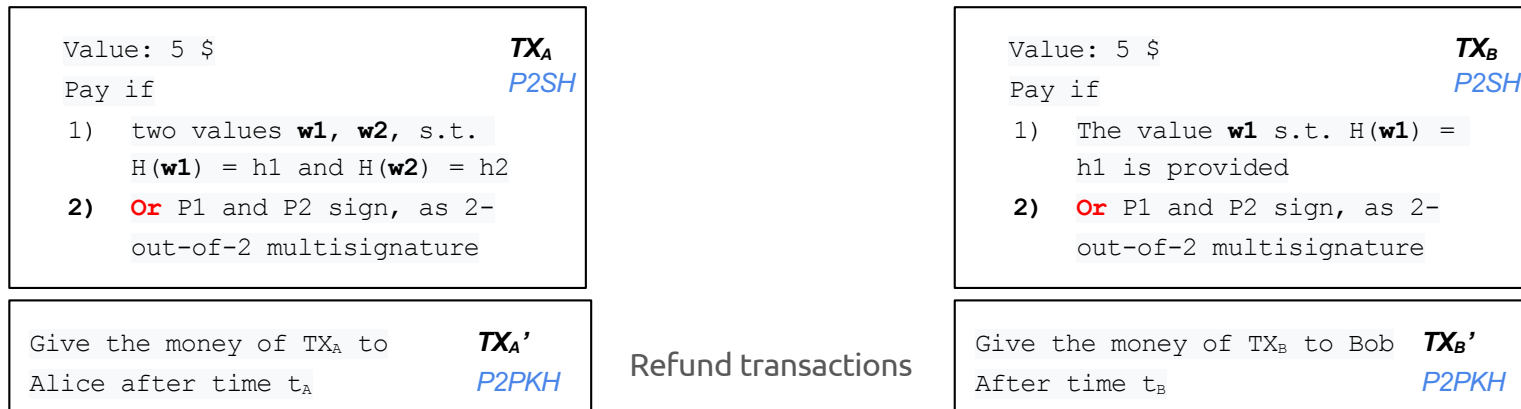
w₂, h₁=H(w₁)



Fair swap of values using time-locks, Setup



Fair swap of values using time-locks, Setup



Fair swap of values using time-locks, Execution

- P_1 :
 - Creates a P2SH transaction TX for \$X provided that:
 - i. (P_1 and P_2 sign, as 2-out-of-2 multisignature) *or*
 - ii. (P_2 signs and reveals $\mathbf{w}_1, \mathbf{w}_2$, s.t. $H(\mathbf{w}_1) = \mathbf{h}_1$ and $H(\mathbf{w}_2) = \mathbf{h}_2$)
 - Creates a P2PKH transaction TX' that spends the output of TX with a time-lock in the near future
 - Sends TX' to P_2 to sign it (P_2 does not see TX, only the tx id is needed to refer to it)
- P_2 acts in the same way:
 - Create a TX that can be redeemed via (2-out-of-2 multisig) or (P_1 signs and reveals $\mathbf{w}_1$, s.t. $H(\mathbf{w}_1) = \mathbf{h}_1$)
 - Create a corresponding time-locked TX' and send to P_1 to sign
- Completion:
 - P_1 publishes its TX, so P_2 can redeem \$X by revealing $\mathbf{w}_1, \mathbf{w}_2$
 - P_2 publishes its TX, so P_1 can redeem \$X by revealing $\mathbf{w}_1$
 - P_1 reveals $\mathbf{w}_1$ and redeems \$X (from P_2 's TX)
 - P_2 reveals $\mathbf{w}_1, \mathbf{w}_2$ and redeems \$X (from P_1 's TX)
- If either party aborts, the other can claim \$X (from their TX) after time-lock fires, by publishing their TX'

Pay to script hash (P2SH)

Pay-to-Public-Key-Hash (P2PKH)

Fair swap of values using time-locks, Notes

- If P_1 's TX could be redeemed by " $H(\mathbf{w}_2) = \mathbf{h}_2$ and P_2 signs it":
 - P_2 could reveal $\mathbf{w}_2$ and obtain payment of $\$X$, without publishing its own TX transaction
 - P_1 would obtain the output $\mathbf{w}_2$ but lose $\$X$
 - (note that we cannot ensure that the TX transactions will appear concurrently in the blockchain)
- If a multisig was not used for the refunds, a player could:
 - Submit its value
 - Rush to obtain its refund, invalidating the TX payment of the other player
- The time-lock for P_1 should be less than that for P_2 ; if equal, P_1 could:
 - Wait for the very last minute to reveal $\mathbf{w}_1$
 - Hope that time-lock fires before P_2 can publish $\mathbf{w}_2$ on the chain
 - Claim $\$X$ even if P_2 tries to act honestly (and reveals $\mathbf{w}_2$ out of time)

Fair Computation

- The two parties use MPC to compute a secret sharing of the output of the computation
 - $w_1 + w_2 = \text{MPC_output}$
- Subsequently parties do a fair swap of values, to obtain the MPC_output:
 - If a party aborts, the other will be compensated

N-party ladder construction, I

- Uses N-out-of-N multisig for refunds
- P_N can redeem $\$X$ from each player if it reveals $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_N$ (i.e., the N-1 parties prepare these “roof” **TX** transactions)
- For $i = 1, \dots, N-1$, player P_{N-i} can redeem from player P_{N-i+1} an amount equal to $\$X(N-i)$ if it reveals $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{N-i}$ (the N-1 parties also prepare these “ladder” **TX** transactions)
- Redeeming follows the sequence $P_1, P_2, \dots, P_N$

N-party ladder construction, II

- P_1 will redeem $\$X$ from P_2 for publishing $\mathbf{w}_1$
- P_2 will redeem $\$2X$ from P_3 for publishing $\mathbf{w}_1, \mathbf{w}_2$
- ...
- P_{N-1} will redeem $\$(N-1)X$ from P_N for publishing $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{N-1}$
- P_N will redeem $\$X$ from each of $P_1, \dots, P_{N-1}$ for publishing $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_N$

Note before P_N acts, P_1 is at $\$X$, P_2 is at $-\$X + \$2X = \$X$, P_3 is at $-\$2X + \$3X = \$X$, etc.

Post-Quantum Security

=

How to secure a classical
system against quantum attackers?

Schrödinger's Cat

- A cat is put in a sealed box along with a bottle of poison and a radioactive source
- A mechanism releases the poison when it detects radioactive decay
- Is the cat alive or dead?
 - The state of the cat corresponds to the state of the radioactive source
 - The radioactive source is in a superposition of decay and not-decayed
 - The cat is in a superposition of life and death
 - Once we open the box, the outside world get entangles with the inside world: the state collapses into either an alive cat or a dead cat

Quantum Computers

- Quantum computers can perform a computation on a superposition of all possible inputs obtaining a superposition of all outputs (ex. It can evaluate the PoW function on the superposition of all counters at the cost of a single evaluation)
- However, to extract classical information from the quantum state, we need to measure! When we measure the quantum state “collapses”.
- We extract a limited amount of information, but **we lose all the rest!**
 - The outcome of the measurement is described by a probability distribution
 - If we try to repeat the measurement, we obtain the same result as before!
 - Quantum states cannot be copied!

Quantum Computers vs Classical Computers

- Advantages of Quantum Computers
 - They can perform all computations that are possible on a classical computer
 - Some problems can be solved more efficiently on quantum computers (in theory)
- Advantages of Classical Computers
 - We can perform classical computations at scale
 - current quantum computers (if we can call them so) can only evaluate very simple circuits and at huge costs

Quantum Advantage

Three classes of problems where quantum computers are better:

- **Search algorithms (mildly better):** given a database of N pairs x and a property P , return (at least) an entry that satisfies P
- **Period finding (much better):** given a periodic function f , determine the period T
- **Quantum simulation (typically relevant for physicists, chemists and biologist, not so much for us):** model quantum phenomena (e.g. 3D-structure of molecules)

Grover's Search

Given a database containing N values x and a property P , find (at least one) x such that $P(x)=1$.

- **Classically:** (unless we are lucky) we need $O(N)$ computation
- **Quantumly:** we just need $O(\sqrt{N})$ computation

Grover's Search

Consequences:

- If a signature scheme has a 128-bit secret key, a quantum computer can recover it from just the public key in time $O(2^{64})$, a classical algorithm would need time $O(2^{128})$
- If H is a hash function with 256-bit output, a quantum computer can find a collision in time $O(2^{64})$, a classical algorithm would need time $O(2^{128})$
- If we use H for a PoW-based distributed ledger and the hardness parameter is T , a quantum computer can mine the next block in time $\sim \sqrt{2^{256}/T}$. A classical computer requires time $\sim 2^{256}/T$.

Grover's Search

Conclusion: If we want to design post-quantumly secure systems with the same level of security that we currently achieve against classical attackers, we need to double the size of all keys and hashes

Ex: When/ if quantum computers become a reality:

- We should use 256-bit signature keys
- We should use hash functions with 512-bit outputs
- We should square the hardness parameter T of a PoW and use a 512-bit hash function

Why post-quantum security today?

In some contexts, it is wise to increase the key size today

- an attacker can store today's encrypted communication to decrypt it in the future, when/if quantum computers will become a reality.
- It could try to wait for the advent of quantum computers to forge today's signatures (it may try to rewrite history)

Quantum Fourier Transform

Given a function $F: \mathbb{Z}_N \rightarrow D$, find the minimum T such that $F(x+T)=F(x)$ for every x in $\mathbb{Z}_N$.

- **Classically:** (unless we are lucky) we need $O(N)$ computation
- **Quantumly:** we just need $\text{polylog}(N)$ computation

Why do we care?

- A lot of cryptography is based on periodic functions (ex: Diffie-Hellman)
- There are strong connections between the problem of period finding and factoring (ex. RSA). **If we can find T , we can factor N .**

Quantum Fourier Transform

Some cryptographic tools are **completely** insecure against quantum attackers:

- Some public-key encryption schemes (ex. RSA, Diffie-Hellman)
- Some signature schemes (ex. ECDSA, EdDSA, RSA)
- Most VRFs and fancy signatures we have seen in Lecture 8 (all pairing-based cryptography is not post-quantum)

Some constructions are still secure (as long as we double the key/hash size):

- most hash functions (ex. SHA2 and SHA3)
- most symmetric encryption schemes (ex. AES)

Quantum Fourier Transform

Cryptography is based on the hardness of solving certain problems.

Popular examples:

- **Factoring:** given a large random integer N , it is hard to find its factorisation
- **Discrete logarithm:** given three random integers h , g and p , where p is a large prime, it is hard to find an integer x such that $g^x \bmod p = h$
- RSA is based on the hardness of factoring
- Diffie-Hellman, ECDSA and EdDSA are based on the hardness of computing the discrete logarithm (typically over elliptic curves)

Quantum Fourier Transform

We need to substitute old cryptographic tools with new ones that are based on quantumly hard problems.

Examples:

- **Lattice-based cryptography:** based on the hardness of Learning with Errors (LWE) or the Shortest Integer Solution (SIS)
- **Code-based cryptography:** based on the hardness of Learning Parity with Noise (LPN)
- **Multivariate quadratic cryptography:** based on the hardness of finding solutions to systems of multivariate polynomial equations of degree 2
- **Isogeny-based cryptography:** based on the hardness of inverting particular maps called isogenies

Quantum Fourier Transform

There is an ongoing standardization process for post-quantum cryptographic tools (organised by NIST).

Selected public-key encryption schemes (to substitute RSA and Diffie-Hellman):

- ML-KEM/Kyber (based on lattices)

Selected signature schemes (to substitute ECDSA and EdDSA):

- ML-DSA/Dilithium (based on lattices)
- SLH-DSA/ SPHINCS+ (based on hash functions)
- FN-DSA/Falcon (based on lattices)

These solutions achieve similar efficiency to that of their quantumly broken counterparts.

References

- For secret sharing and multi-party computation in general, look at Chapter 3, until Section 3.3.2 of the following book (you can access to the book with your university account).
 - Cramer, R., Damgård, I., & Nielsen, J. (2015). *Secure Multiparty Computation and Secret Sharing*. Cambridge: Cambridge University Press. doi:10.1017/CBO9781107337756.
- For fair swap, and in particular for how to achieve fairness with compensation in multi-party computation, please look at this paper and follows the references when something is not clear.
 - Marcin Andrychowicz, Stefan Dziembowski, Daniel Malinowski, and Lukasz Mazurek. Fair two-party computations via the bitcoin deposits. In 1st Workshop on Bitcoin Research 2014 (in Association with Financial Crypto), 2014. <http://eprint.iacr.org/2013/837>.